



THE UNIVERSITY OF
WAIKATO
Te Whare Wānanga o Waikato

COMPX527-25B (HAM) – Secure Cloud Application Engineering

PROJECT PROPOSAL

WORKING TITLE: **FLEXSHARE**

Student Name	Student ID	Roles
Ting Tang	1686923	Front – End
Hongwei Ding	1660307	Front – End
Gita Ranmal Modhvadia	1673767	Security/Group Leader
Muhammad Ali Javed	1679215	Security
Justin Poutoa	1620107	Back – End
Leon Liang	1682536	Back - End

OVERVIEW

This project aims to develop a secure, cloud-based ride-sharing web application that combines the affordability and predictability of public transport with the flexibility of peer-to-peer ride services. Inspired by New Zealand's Flex bus service and ride-hailing platforms like Uber, our solution introduces a hybrid transportation model where riders are picked up from any location but dropped off only at designated bus stop zones and vice versa. This approach lowers costs, improves scheduling efficiency, and makes shared rides more sustainable and accessible.

Unlike traditional public transport, which is often limited in availability, our application offers 24/7 peer-to-peer rides to fixed stops, ensuring both affordability and wider coverage.

Built on Amazon Web Services (AWS), the project leverages cloud-native infrastructure to ensure high availability, scalability, and strong security controls. Both drivers and riders will access the service through a shared, responsive web application, reducing platform fragmentation and increasing reach. Security, privacy, and efficiency are prioritized using modern AWS services like EC2, Cognito, DynamoDB.

The result is a cost-effective, secure, and scalable transportation alternative designed specifically for the New Zealand context, with potential to scale globally.

PROBLEM STATEMENT

- Public transport services like New Zealand's Flex are limited, operating only on specific days and with restricted time windows, leaving many users without timely transport options.
- Traditional rideshare platforms such as Uber are often too expensive and they offer little incentive for ride pooling, making them inaccessible for cost-sensitive users.
- There is currently no hybrid solution that combines the convenience of peer-to-peer rides with the structured efficiency of fixed-route stops.

Additionally, private cars consume excessive fuel and contribute to air pollution. A system that allows users to spend few time setting routes and share rides with others at a low cost would be beneficial to everyone.

DATASET SELECTION

DATASET SELECTED: Hamilton City Traffic Volumes

REFERENCE: <https://hamilton.govt.nz/your-city/data-and-statistics/understanding-our-city/traffic-volumes>

The Hamilton City Traffic Volumes dataset provides real-world vehicle flow data, enabling optimized route planning for our ride-share service to reduce travel time and operational costs.

SERVICES & REQUEST FOR SERVICES

TOTAL BUDGET ESTIMATION: ~ \$30 NZD

SERVICES	JUSTIFICATION
S3	Stores front-end resources and sensitive driver photos, such as images of driver's licenses [10].
DynamoDB	Stores order information, historical GPS tracks, and other frequently accessed data [4].
EC2 (4 Instances)	2 instances for general backend services [6]. 2 instances for hosting MySQL containers [6].
Elastic Load Balancing (ELB)	Distributes traffic evenly between the two backend EC2 instances [7].
CloudWatch	Collect and store logs, trigger alarms on defined thresholds, and provide dashboards for real time monitoring [3].
IAM	- Manages users, groups, roles, and permissions [8]. - Enforce fine grained, least privilege access control [8].
SQS	Handles GPS data caching and queuing [12].
Secrets Manager	Secures and encrypts API keys and passwords [11].
CloudFront	Accelerates delivery of resources stored in S3 [2].

REQUEST FOR SERVICES

SERVICES	JUSTIFICATION
API Gateway	Monitors Traffic, CORS configuration, and manages API endpoints [1].
AWS Cognito	Handles user registration, login, and access control at scale [5].

TIMELINE OF THE PROJECT

ROLES/WEEKS	WEEK 1 (04 AUG)	WEEK 2 (11 AUG)	WEEK 3 (18 AUG)	WEEK 4 (25 AUG)	WEEK 5 (01 SEP)	WEEK 6 (08 SEP)	WEEK 7 (15 SEP)
PROPOSAL							
DESIGN							
DEVELOPMENT							
TESTING							
DEMONSTRATION							

CHALLENGES OF THE PROJECT

- **Google Maps API Integration:** Google Maps API can be difficult to manage due to quota limits and front-end performance issues. Management: implement caching of static map tiles; monitor usage via CloudWatch; request quota increases early.
- **Route Optimization for Multi-Riders:** Multi-rider, multi-stop optimization is computationally hard. Management: adopt heuristic algorithms (e.g., Google OR-Tools); benchmark on sample data; parallelize tasks via Lambda only for compute bursts.
- **Authentication & Roles:** Improper Cognito or IAM settings may break functionality across drivers and riders. Management: enforce least-privilege IAM roles; use Cognito groups; review policies bi-weekly; automated policy linter in CI pipeline.
- **Testing Without Deployment:** Without live deployment, simulating real-world traffic, requests, and usage patterns is limited and unreliable. Management: use Docker-based staging with synthetic traffic generators; incorporate Chaos Monkey style fault injection.
- **Team Member Contribution:** If members are not collaborative or not willing to participate, leading to delay of project process. Management: Use approach of group discussion to solve the problem if it does not solve, will talk to supervisor.
- **Time management:** Delay of project due to no proper time management. Management: use weekly meeting approach to know the progress of the project.

REFERENCES

1. Amazon Web Services. (n.d.). *Amazon API Gateway documentation*. AWS.
<https://docs.aws.amazon.com/apigateway/>
2. Amazon Web Services. (n.d.). *Amazon CloudFront documentation*. AWS.
<https://docs.aws.amazon.com/cloudfront/>
3. Amazon Web Services. (n.d.). *Amazon CloudWatch documentation*. AWS.
<https://docs.aws.amazon.com/cloudwatch/>
4. Amazon Web Services. (n.d.). *Amazon Cognito documentation*. AWS.
<https://docs.aws.amazon.com/cognito/>
5. Amazon Web Services. (n.d.). *Amazon DynamoDB documentation*. AWS.
<https://docs.aws.amazon.com/dynamodb/>
6. Amazon Web Services. (n.d.). *Amazon EC2 documentation*. AWS.
<https://docs.aws.amazon.com/ec2/>
7. Amazon Web Services. (n.d.). *Amazon Elastic Load Balancing documentation*. AWS.
<https://docs.aws.amazon.com/elasticloadbalancing/>
8. Amazon Web Services. (n.d.). *Amazon IAM documentation*. AWS.
<https://docs.aws.amazon.com/iam/>
9. Amazon Web Services. (n.d.). *Amazon S3 documentation*. AWS.
<https://docs.aws.amazon.com/s3/>
10. Amazon Web Services. (n.d.). *Amazon Secrets Manager documentation*. AWS.
<https://docs.aws.amazon.com/secretsmanager/>
11. Amazon Web Services. (n.d.). *Amazon Simple Queue Service (SQS) documentation*. AWS.
<https://docs.aws.amazon.com/sqs/>
12. Hamilton City Council. (n.d.). *Traffic volumes*. <https://hamilton.govt.nz/your-city/data-and-statistics/understanding-our-city/traffic-volumes>